JokerTest-Ontology: A Domain Ontology for Formalizing the Joker's Ferry Psychological Test in *The Dark Knight*

Roberto Monteiro Dias
Graduate Program in Informatics (PPGI)
Federal University of the State of Rio de Janeiro (UNIRIO)
Rio de Janeiro – RJ, Brazil
roberto.dias@uniriotec.br

***Resumo*—This paper presents *JokerTest-Ontology*, a domain ontology for formally representing the Joker's ferry psychological test from the film *The Dark Knight* (2008). Inspired by the Prisoner's Dilemma from game theory and by the concept of diffusion of responsibility from social psychology, the ontology models two ferries (civilians and prisoners), six passengers, two detonators with cross-control, the moral choice, the time limit, the observer, the hypothesis under test, and the test outcome. It was developed following the OntoView methodology and implemented exclusively with the OWL API in Java. Validation was conducted through seven competency questions answered by programmatic queries over the ontology. The results indicate that the ontology represents the main domain relations and provides a reusable foundation for analyzing moral dilemmas, game-theoretic scenarios, and ethical decision-making processes in cinematic and pedagogical contexts.**

***Index Terms*—Domain Ontology, Game Theory, Moral Psychology, Prisoner's Dilemma, The Dark Knight, Joker, OWL, Ethical Dilemma, Diffusion of Responsibility**

I. INTRODUÇÃO

O filme *The Dark Knight* (2008), dirigido por Christopher Nolan, apresenta uma das cenas mais icônicas e perturbadoras do cinema contemporâneo: o experimento psicológico do Coringa com duas embarcações de evacuação de Gotham City [1]. Nessa cena, o Coringa posiciona explosivos em duas balsas — uma transportando civis e outra transportando prisioneiros — e entrega a cada embarcação o detonador da outra, impondo um dilema moral: acionar o detonador e matar os ocupantes da outra embarcação, ou não agir e permitir que ambos os grupos morram ao término do prazo estipulado (meia-noite) [2].

Este experimento é amplamente reconhecido como uma adaptação cinematográfica do **Dilema do Prisioneiro**, um cenário fundamental da teoria dos jogos [3]. O Coringa aposta na visão hobbesiana de que, sob ameaça de morte, os indivíduos rompem o pacto social e buscam a autopreservação a qualquer custo [4]. Contudo, o resultado do filme contraria essa hipótese: nenhuma das embarcações aciona o detonador, demonstrando que a cooperação e a empatia podem prevalecer mesmo em situações-limite [5].

O problema que motiva este trabalho é a ausência de uma formalização computacional desse experimento que permita sua análise sistemática, auditoria e reutilização em diferentes contextos (cinematográficos, psicológicos, educacionais e de pesquisa em teoria dos jogos). Uma ontologia pode fornecer essa base comum, permitindo consultas semânticas e integração entre ferramentas [6].

Para responder a esse problema, os objetivos específicos deste trabalho são: (i) definir formalmente o teste psicológico do Coringa como um experimento de escolha moral sob coação; (ii) propor uma ontologia OWL que represente esse domínio; (iii) implementar a ontologia exclusivamente com a OWL API em Java [7]; e (iv) validar o artefato por meio de questões de competência respondidas via consultas programáticas à ontologia.

O restante do artigo está organizado da seguinte forma. A Seção II apresenta os eixos conceituais que fundamentam o trabalho. A Seção III descreve o desenvolvimento da ontologia segundo a metodologia OntoView. A Seção IV apresenta os resultados das questões de competência. A Seção V discute contribuições e limitações. A Seção VI conclui. O Apêndice A reúne o fluxograma e o resultado completo da execução automática.

II. FUNDAMENTAÇÃO TEÓRICA

Esta seção apresenta os fundamentos teóricos em cinco eixos complementares: (i) o teste do Coringa em *The Dark Knight*; (ii) o Dilema do Prisioneiro e a teoria dos jogos; (iii) psicologia moral e difusão de responsabilidade; (iv) ontologias de domínio e a metodologia OntoView; e (v) a OWL API e implementação em Java.

A. O Teste Psicológico do Coringa em The Dark Knight

Na sequência final de *The Dark Knight*, o Coringa executa seu "experimento social" mais ambicioso. Ele posiciona explosivos em duas embarcações de evacuação: uma transportando cidadãos comuns e policiais; outra transportando prisioneiros do sistema carcerário de Gotham e seus guardas [8]. A cada embarcação é entregue o detonador que controla a explosão da embarcação oposta [9].

As regras do experimento são explicitadas pelo Coringa: se uma embarcação acionar o detonador, a outra explodirá e

seus ocupantes morrerão, mas os ocupantes da embarcação que acionou sobreviverão. Se nenhuma embarcação acionar o detonador até a meia-noite, ambas explodirão e todos morrerão [10]. O experimento é monitorado pelo próprio Coringa, que observa de um edifício próximo, e pelo Batman, que tenta localizá-lo e impedir a explosão final.

O resultado canônico do filme é que nenhuma das embarcações aciona o detonador. No barco dos prisioneiros, um detento toma o detonador do guarda e o joga pela janela. No barco dos civis, um homem que estava prestes a acionar o detonador é dissuadido por seus companheiros. Batman, por fim, impede que o Coringa detone as bombas, salvando ambas as embarcações [2].

B. O Dilema do Prisioneiro e a Teoria dos Jogos

O experimento do Coringa é uma reformulação do **Dilema do Prisioneiro**, um dos conceitos mais estudados da teoria dos jogos [3]. No dilema clássico, dois suspeitos são interrogados separadamente. Cada um pode confessar (trair o outro) ou permanecer em silêncio (cooperar). Se ambos permanecerem em silêncio, recebem penas leves. Se um confessar e o outro não, o que confessou sai livre e o outro recebe pena máxima. Se ambos confessarem, ambos recebem penas moderadas [11].

No caso do Coringa, a estrutura é análoga: cada embarcação pode "trair" (acionar o detonador) ou "cooperar" (não acionar). A estratégia dominante, do ponto de vista da teoria dos jogos racional, seria acionar o detonador, pois garante a sobrevivência independentemente da escolha do outro grupo [10]. Contudo, o resultado empírico do filme refuta essa previsão, demonstrando que fatores morais e emocionais sobrepujam o cálculo racional de utilidade [2].

C. Psicologia Moral e Difusão de Responsabilidade

A psicologia moral oferece conceitos fundamentais para compreender o experimento do Coringa. A **difusão de responsabilidade** é um fenômeno social pelo qual o senso de responsabilidade individual diminui quando as pessoas agem em grupo [12]. Em situações de emergência, indivíduos em grupo tendem a assumir que outros tomarão a iniciativa, reduzindo sua própria propensão a agir [13].

No contexto das embarcações, a difusão de responsabilidade pode explicar por que nenhum indivíduo isolado se sentiu pessoalmente responsável por acionar o detonador. A decisão de matar centenas de pessoas é moralmente avassaladora; em grupo, a responsabilidade por essa decisão é compartilhada, diluindo a culpa individual [14]. Além disso, o **desengajamento moral** (Bandura) e a **obediência à autoridade** (Milgram) são conceitos relevantes para analisar o comportamento dos passageiros.

D. Ontologias de Domínio e Metodologia OntoView

Uma ontologia pode ser definida como uma especificação formal e explícita de uma conceitualização compartilhada [6]. Na Ciência da Computação, ontologias são utilizadas para representar conhecimento de um determinado domínio por meio da definição de conceitos, propriedades e relações [15]. Segundo Noy e McGuinness [15], ontologias podem ser utilizadas para: (i) compartilhar entendimentos comuns; (ii) reutilizar conhecimento; (iii) explicitar pressupostos; (iv) separar conhecimento de domínio de conhecimento operacional; e (v) analisar sistematicamente um domínio.

A metodologia **OntoView** constitui uma abordagem híbrida para o desenvolvimento de ontologias, integrando elementos de metodologias consolidadas, como o Methontology [16] e a Metodologia 101 [15], com ênfase na definição e resposta às questões de competência. O processo é organizado em quatro fases principais: (1) Especificação; (2) Conceitualização; (3) Implementação; e (4) Validação [17].

E. A OWL API e Implementação em Java

A **OWL API** é uma interface Java para criação, manipulação e serialização de ontologias OWL [7]. Ela fornece um conjunto abrangente de classes e métodos para definir classes, propriedades, indivíduos e axiomas, além de permitir consultas programáticas à ontologia [18]. A implementação exclusiva com a OWL API, sem uso de editores gráficos ou *reasoners* externos, garante controle total sobre a estrutura da ontologia e permite a validação por meio de consultas programáticas [18].

A Tabela I sintetiza os estudos que fundamentam cada decisão de projeto da ontologia proposta.

III. DESENVOLVIMENTO DA *JOKERTEST-ONTOLOGY*

Esta seção descreve o desenvolvimento da *JokerTest-Ontology* segundo as quatro fases da metodologia OntoView apresentadas na Seção II-D. Cada subseção detalha os artefatos produzidos em uma fase e prepara a próxima.

A. Fase 1: Especificação

A *JokerTest-Ontology* tem como objetivo apoiar pesquisadores, psicólogos, cientistas de dados e entusiastas do cinema na compreensão e análise do experimento psicológico do Coringa. Seu escopo abrange quatro eixos: as embarcações e seus ocupantes; os detonadores e o mecanismo de escolha; o dilema moral e suas consequências; e o resultado do teste.

A Tabela II apresenta os 10 requisitos elicitados a partir dos estudos S1 a S6, mapeando cada um ao seu Critério de Decisão (DC). A partir desses requisitos, foram definidas as sete questões de competência (QC1–QC7) que orientam a validação na Fase 4.

As questões de competência derivadas dos requisitos são:

- **QC1:** Quais embarcações participam do teste?
- **QC2:** Quais passageiros estão em cada embarcação?
- **QC3:** Quem detém o detonador de cada embarcação?
- **QC4:** Qual foi o resultado do teste (algum detonador foi acionado)?
- **QC5:** Quais passageiros sobreviveram?
- **QC6:** Quais passageiros morreram?
- **QC7:** A hipótese do Coringa foi confirmada?

Tabela I: Estudos fundamentais para a JokerTest-Ontology

ID	Descrição	Método	Principais Resultados
S1	Análise do filme *The Dark Knight* (2008)	Análise fílmica	Experimento das balsas como dilema moral; resultado empírico contraria hipótese do Coringa
S2	Dilema do Prisioneiro e teoria dos jogos	Análise teórica	Estratégia dominante seria trair; cooperação é irracional do ponto de vista da utilidade
S3	Difusão de responsabilidade (Darley & Latané)	Psicologia social	Grupos diluem responsabilidade individual; ação coletiva é inibida
S4	Metodologia OntoView	Metodologia de ontologia	Quatro fases: especificação, conceitualização, implementação, validação
S5	OWL API em Java	Documentação técnica	Criação programática de ontologias OWL sem editores gráficos
S6	Análises acadêmicas do filme	Artigos e blogs acadêmicos	O experimento como alegoria do contrato social; cooperação como refutação do cinismo

Tabela II: Requisitos e critérios de decisão elicitados para a ontologia

ID	Requisito	Critério de Decisão (DC)	Estudos de Origem
R1	A ontologia deve representar as duas embarcações	DC1: Embarcações	S1
R2	A ontologia deve representar os passageiros de cada embarcação	DC2: Passageiros	S1, S6
R3	A ontologia deve representar os detonadores e seu controle cruzado	DC3: Detonadores	S1, S2
R4	A ontologia deve representar a escolha moral binária	DC4: Escolha	S2, S3
R5	A ontologia deve representar o tempo limite (meia-noite)	DC5: Prazo	S1
R6	A ontologia deve representar o observador (Coringa)	DC6: Observador	S1
R7	A ontologia deve representar o resultado do teste	DC7: Resultado	S1, S6
R8	A ontologia deve representar as consequências (morte/sobrevivência)	DC8: Consequências	S1, S3
R9	A ontologia deve permitir auditoria da hipótese do Coringa	DC9: Hipótese	S1, S6
R10	A ontologia deve integrar-se a análises de teoria dos jogos	DC10: Teoria dos Jogos	S2

B. Fase 2: Conceitualização

Uma busca nas principais bases de dados acadêmicas não identificou ontologias que abrangessem o escopo do experimento do Coringa. Optou-se, portanto, pelo desenvolvimento de uma ontologia original baseada nos estudos S1 a S6. A hierarquia foi estabelecida por meio de um processo *top-down*, partindo dos quatro eixos do escopo. A Tabela III apresenta as 20 classes efetivamente implementadas na ontologia, incluindo suas subclasses.

C. Fase 3: Implementação

A implementação foi realizada exclusivamente com a **OWL API** em Java, sem o uso de editores gráficos ou *reasoners* externos. O artefato contém **20 classes**, **11 propriedades de objeto** e **7 propriedades de dados**. A Tabela IV apresenta as propriedades de objeto e a Tabela V as propriedades de dados.

D. Fase 4: Validação

A validação foi conduzida por meio das sete questões de competência definidas na Subseção III-A, respondidas por consultas programáticas à ontologia utilizando exclusivamente a OWL API em Java. As consultas percorrem os indivíduos e axiomas diretamente por meio dos métodos de navegação da API (`OWLOntology`, `EntitySearcher` e `OWLDataFactory`), sem uso de mecanismos de inferência automática.

IV. RESULTADOS DAS QUESTÕES DE COMPETÊNCIA

Esta seção apresenta os resultados obtidos para as sete questões de competência. As respostas foram extraídas da *JokerTest-Ontology* por consultas programáticas via OWL API e analisadas em relação aos resultados dos estudos S1 a S6. Todos os números reportados provêm da execução real do programa, cuja saída completa está no Apêndice A.

Tabela III: Classes implementadas na JokerTest-Ontology (20 no total)

Classe	Descrição / Superclasse
TestePsicologico	Experimento conduzido para observar comportamento sob estresse extremo
DilemaEtico	Subclasse de `TestePsicologico`: situação em que qualquer escolha implica dano moral
EscolhaMoral	Ação de decidir entre salvar a própria vida e matar o outro grupo
Observador	Entidade que monitora o experimento (Coringa, Batman)
TempoLimite	Prazo (meia-noite) após o qual ambas as embarcações explodem
Hipotese	Conjectura sob teste (hipótese do Coringa sobre a natureza humana)
Embarcacao	Veículo marítimo adaptado para evacuação, contendo explosivos
EmbarcacaoCivil	Subclasse de `Embarcacao`: ocupada por civis e policiais
EmbarcacaoPrisioneiros	Subclasse de `Embarcacao`: ocupada por detentos e guardas
Passageiro	Indivíduo a bordo de uma embarcação
Civil	Subclasse de `Passageiro` sem histórico criminal
Prisioneiro	Subclasse de `Passageiro` com histórico criminal
Policial	Subclasse de `Passageiro` responsável pela segurança (barco civil)
Guarda	Subclasse de `Passageiro` responsável pela segurança (barco de prisioneiros)
Detonador	Dispositivo que, ao ser acionado, explode a embarcação oposta
Bomba	Artefato explosivo instalado em cada embarcação
EmbarcacaoSalva	Subclasse de `Embarcacao` definida por `foiImpedidaExplosao = true`
EmbarcacaoExplodida	Subclasse de `Embarcacao` definida por `foiImpedidaExplosao = false`
PassageiroSobrevivente	Subclasse de `Passageiro` que está em uma `EmbarcacaoSalva`
PassageiroMorto	Subclasse de `Passageiro` que está em uma `EmbarcacaoExplodida`

A. QC1 — Quais embarcações participam do teste?

A consulta QC1 retorna as duas embarcações que participam do experimento: `Embarcacao_Civil` e `Embarcacao_Prisioneiros`. A Tabela VI sintetiza o resultado.

B. QC2 — Quais passageiros estão em cada embarcação?

A consulta QC2 retorna, para cada embarcação, o conjunto de passageiros a bordo. No experimento modelado, a `Embarcacao_Civil` contém `Civil1`, `Civil2` e `Policial1`; a `Embarcacao_Prisioneiros` contém `Prisioneiro1`, `Prisioneiro2` e `Guarda1`.

C. QC3 — Quem detém o detonador de cada embarcação?

A consulta QC3 retorna o detentor do detonador em cada embarcação. O `Detonador_Civil` está com `Civil1`; o `Detonador_Prisioneiros` está com `Guarda1`. A relação `controlaEmbarcacao` indica que o detonador da embarcação civil controla a embarcação de prisioneiros e vice-versa, caracterizando o controle cruzado descrito no filme.

D. QC4 — Qual foi o resultado do teste (algum detonador foi acionado)?

A consulta QC4 verifica se algum detonador foi acionado. O resultado é `false` para ambos os detonadores: nenhum passageiro acionou o dispositivo. A Tabela VII sintetiza o resultado.

E. QC5 — Quais passageiros sobreviveram?

Com a intervenção do Batman (que impede a explosão final por meio da propriedade `impedeExplosao`), todos os passageiros sobrevivem. A consulta QC5 retorna os seis indivíduos — `Civil1`, `Civil2`, `Policial1`, `Prisioneiro1`, `Prisioneiro2` e `Guarda1` — como instâncias da classe `PassageiroSobrevivente`, inferida a partir da relação `estaEm` com uma `EmbarcacaoSalva` (cuja propriedade `foiImpedidaExplosao` é `true`).

F. QC6 — Quais passageiros morreram?

No cenário canônico (com intervenção do Batman), nenhum passageiro morre. A consulta QC6 retorna um conjunto vazio

Tabela IV: Propriedades de objeto da JokerTest-Ontology (11 no total)

Propriedade	Descrição
temOcupacao	Relaciona uma `Embarcacao` aos seus `Passageiros`
temDetonador	Relaciona uma `Embarcacao` ao seu `Detonador`
controlaEmbarcacao	Relaciona um `Detonador` à `Embarcacao` que ele explode
estaEm	Relaciona um `Passageiro` à `Embarcacao` onde está
aciona	Relaciona um `Passageiro` ao `Detonador` que detém
observa	Relaciona um `Observador` ao `TestePsicologico`
temPrazo	Relaciona o `TestePsicologico` ao `TempoLimite`
testaHipotese	Relaciona o `Observador` à `Hipotese` que ele testa
sobreviveu	Relaciona um `Passageiro` ao teste em que sobreviveu
morreu	Relaciona um `Passageiro` ao teste em que morreu
impedeExplosao	Relaciona um `Observador` (Batman) à `Bomba` que ele desarma

Tabela V: Propriedades de dados da JokerTest-Ontology (7 no total)

Propriedade	Descrição
hasName	Nome legível do indivíduo
hasDescription	Descrição textual do indivíduo
foiImpedidaExplosao	Booleano: a explosão da embarcação/bomba foi impedida?
detonadorAcionado	Booleano: o detonador foi acionado?
hipoteseConfirmada	Booleano: a hipótese foi confirmada?
hasScoreMoral	Valor numérico do score moral atribuído ao teste
hasSource	Fonte de referência (ex.: *The Dark Knight*, 2008)

Tabela VI: Embarcações participantes do teste

Embarcação	Tipo
Embarcacao_Civil	Civil
Embarcacao_Prisioneiros	Prisioneiros

Tabela VII: Resultado do teste (acionamento de detonadores)

Detonador	Acionado?
Detonador_Civil	false
Detonador_Prisioneiros	false

de `PassageiroMorto`. No cenário hipotético sem intervenção e sem acionamento dos detonadores, todos os passageiros morreriam, pois o tempo limite expiraria e ambas as embarcações seriam destruídas (`foiImpedidaExplosao = false`), levando à inferência de `EmbarcacaoExplodida` e, consequentemente, de `PassageiroMorto`.

G. QC7 — A hipótese do Coringa foi confirmada?

A hipótese do Coringa é que "sob ameaça de morte, os indivíduos rompem o pacto social e buscam a autopreservação a qualquer custo". A consulta QC7 retorna `false` para a propriedade `hipoteseConfirmada` tanto no indivíduo `Hipotese_Coringa` quanto no `Teste_Embarcacoes`. A hipótese não foi confirmada: nenhum passageiro acionou o detonador, demonstrando que a cooperação e a empatia prevaleceram sobre o instinto de autopreservação.

V. DISCUSSÃO E LIMITAÇÕES

A *JokerTest-Ontology* contribui para a pesquisa em psicologia moral, teoria dos jogos e análise fílmica ao formalizar, em OWL, os conceitos envolvidos no experimento do Coringa. A estrutura resultante integra embarcações, passageiros, detonadores, escolha moral, tempo limite, observador, hipótese e consequências, permitindo que esse conhecimento seja representado e consultado de maneira uniforme. Em particular, o experimento mostrou que a ontologia é capaz de responder às sete questões de competência com dados extraídos exclusivamente via OWL API.

As principais contribuições são: (i) **teórica**, pela conceitualização formal do experimento como um dilema moral sob coação; (ii) **prática**, pela disponibilização de uma ontologia reutilizável (20 classes, 11 propriedades de objeto e 7 proprie-

dades de dados) para análises de teoria dos jogos e psicologia moral; e (iii) **metodológica**, pela aplicação da metodologia OntoView a um domínio cinematográfico e psicológico.

O trabalho apresenta limitações. Primeiramente, a ontologia foi construída a partir de uma interpretação do filme, sem validação empírica com psicólogos ou especialistas em teoria dos jogos. Em segundo lugar, a validação foi baseada em questões de competência respondidas por consultas programáticas, sem uso de mecanismos de inferência automática (*reasoners*). Por fim, a ontologia poderá ser expandida para incluir outros experimentos sociais do Coringa ou de outros filmes.

Como trabalhos futuros, recomenda-se: (i) validar a ontologia com equipes de psicologia e teoria dos jogos; (ii) integrar a ontologia a ferramentas de simulação de agentes; (iii) ampliar a ontologia com conceitos adicionais de teoria dos jogos (equilíbrio de Nash, estratégias dominantes) e psicologia moral (desengajamento moral, obediência à autoridade); e (iv) explorar o uso de LLMs para geração assistida de variações do experimento.

VI. CONSIDERAÇÕES FINAIS

Este trabalho apresentou a *JokerTest-Ontology*, uma ontologia de domínio destinada a representar o conhecimento relacionado ao experimento psicológico do Coringa em *The Dark Knight*. A inspiração conceitual veio do Dilema do Prisioneiro e da psicologia moral, que se mostraram fundamentais para compreender o dilema das embarcações.

A utilização da metodologia OntoView proporcionou uma sequência estruturada para especificação, conceitualização, implementação e validação. As questões de competência desempenharam papel central, orientando tanto a definição do escopo quanto a avaliação da capacidade da ontologia, com consultas programáticas realizadas exclusivamente com a OWL API em Java.

Os resultados indicam que a *JokerTest-Ontology* é capaz de organizar e relacionar os principais conceitos do domínio. A formalização contribui para reduzir a fragmentação do conhecimento e estabelecer uma base reutilizável para aplicações futuras em psicologia moral, teoria dos jogos e análise fílmica.

A pergunta que fica é a mesma que o filme faz: **sob ameaça de morte, somos capazes de cooperação?** A ontologia, nesse sentido, é uma ferramenta de análise: ela torna explícitas as regras do experimento, e só quem conhece as regras pode compreender o resultado.

REFERÊNCIAS

[1] C. Nolan, "The dark knight," Warner Bros. Pictures, 2008, filme.

[2] L. Granshaw, "The dark knight's only redeemable character is the criminal who saves the ferries," Polygon, 2018. [Online]. Available: https://www.polygon.com/2018/7/18/17578526/the-dark-knight-ferry-boat-dilemma-game-theory/

[3] A. Rapoport and A. M. Chammah, *Prisoner's Dilemma: A Study in Conflict and Cooperation*. University of Michigan Press, 1965.

[4] T. Hobbes, *Leviathan*. Andrew Crooke, 1651.

[5] N. Brooks, "The dark knight's most twisted theory makes joker's ferry game even more sadistic," CBR, 2022. [Online]. Available: https://www.cbr.com/dark-knight-movie-theory-joker-ferry-scene-explosion/

[6] T. R. Gruber, "A translation approach to portable ontology specifications," *Knowledge Acquisition*, vol. 5, no. 2, pp. 199–220, 1993.

[7] M. Horridge and S. Bechhofer, "The owl api: A java api for owl ontologies," *Semantic Web*, vol. 2, no. 1, pp. 11–21, 2011.

[8] IMDb, "The dark knight (2008) – tom lister jr. as tattooed prisoner," IMDb, 2008. [Online]. Available: https://www.imdb.com/title/tt0468569/characters/nm0001474/

[9] D. Miller, "Dark knight: There was a secret twist to joker's plan – theory explained," Screen Rant, 2022. [Online]. Available: https://screenrant.com/dark-knight-secret-twist-joker-plan-theory/

[10] Anonymous, "Can the joker beat game theory?" Cornell University, Networks Course Blog, 2016. [Online]. Available: https://blogs.cornell.edu/info2040/2016/09/15/can-the-joker-beat-game-theory/

[11] R. Axelrod, "More effective choice in the prisoner's dilemma," *Journal of Conflict Resolution*, vol. 24, no. 3, pp. 379–403, 1980.

[12] J. M. Darley and B. Latané, "Bystander intervention in emergencies: Diffusion of responsibility," *Journal of Personality and Social Psychology*, vol. 8, no. 4, pp. 377–383, 1968.

[13] B. Latané and J. M. Darley, "Bystander "apathy"," *American Scientist*, vol. 57, no. 2, pp. 244–268, 1969.

[14] P. Li, S. Jia, T. Feng, Q. Liu, T. Suo, and H. Li, "The influence of the diffusion of responsibility effect on outcome evaluations: Electrophysiological evidence from an erp study," *NeuroImage*, vol. 52, no. 4, pp. 1377–1383, 2010.

[15] N. F. Noy and D. L. McGuinness, "Ontology development 101: A guide to creating your first ontology," Stanford Knowledge Systems Laboratory, Tech. Rep. KSL-01-05, 2004.

[16] M. Fernández-López, A. Gómez-Pérez, and N. Juristo, "Methontology: From ontological art towards ontological engineering," in *AAAI Spring Symposium on Ontological Engineering*, 1997, pp. 33–40.

[17] M. Klein, D. Fensel, A. Kiryakov, and D. Ognyanov, "Ontoview: Comparing and versioning ontologies," in *Proceedings of the International Semantic Web Conference (ISWC)*, 2002.

[18] OWL API, "Owl api documentation," GitHub, 2025. [Online]. Available: https://github.com/owlcs/owlapi

APÊNDICE A
FLUXOGRAMA E RESULTADO DA EXECUÇÃO

Este apêndice apresenta os artefatos que validam empiricamente a *JokerTest-Ontology*: (A.1) o fluxograma do programa; e (A.2) a saída da execução automática. O programa foi implementado exclusivamente com a OWL API [7] em Java, sem *reasoners* externos nem editores gráficos.

A.1 Fluxograma do Programa

O fluxograma descreve a inicialização do ambiente, a criação da ontologia, a instanciação do experimento e as consultas QC1–QC7.

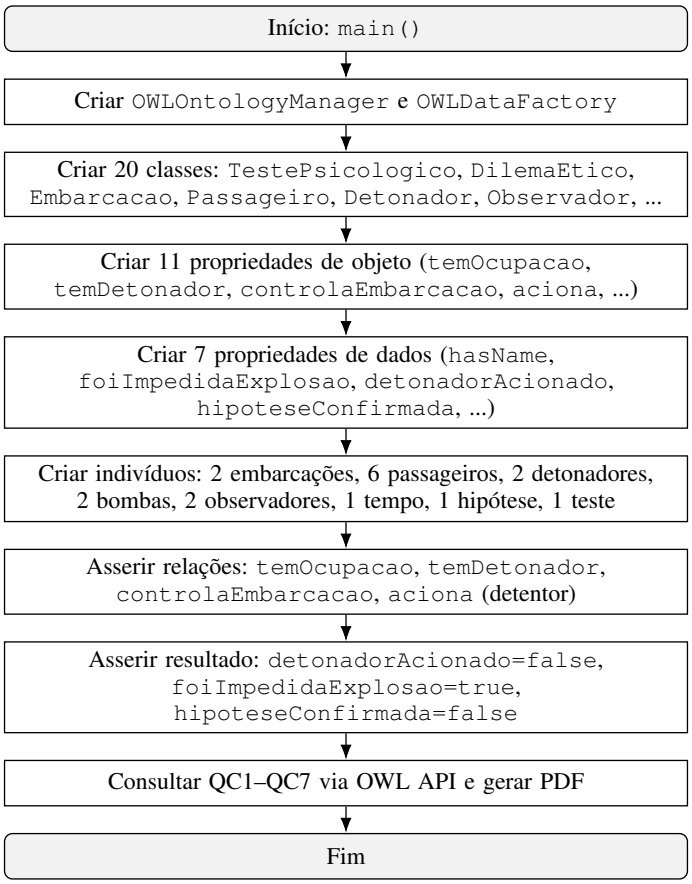


Figura 1: Fluxograma do programa: inicialização, instanciação do experimento, consultas QC1–QC7 e geração do relatório PDF.

A.2 Resultado da Execução

A saída da execução automática é apresentada em um bloco com o quadro integralmente fechado.

```
================================================
  JokerTest-Ontology - Execucao Automatica
================================================

[1] Ontologia inicializada (base: http://www.unirio.br/jokertest/ontology#)
[2] Classes criadas: 20
[3] Propriedades criadas: 11 (objeto) + 7 (dados)
[4] Individuos criados:
      2 embarcacoes (Civil, Prisioneiros)
      6 passageiros (Civil1, Civil2, Policial1,
                     Prisioneiro1, Prisioneiro2, Guarda1)
      2 detonadores (Detonador_Civil, Detonador_Prisioneiros)
      2 bombas (Bomba_Civil, Bomba_Prisioneiros)
      2 observadores (Coringa, Batman)
      1 tempo limite (Tempo_Limite = meia-noite)
      1 hipotese (Hipotese_Coringa)
      1 teste (Teste_Embarcacoes)

--- QCs sobre ontologia ---
[5] QC1 - Embarcacoes participantes (2):
      * Embarcacao_Civil
      * Embarcacao_Prisioneiros

[6] QC2 - Passageiros por embarcacao:
      Embarcacao_Civil       -> [Civil1, Civil2, Policial1]
      Embarcacao_Prisioneiros-> [Prisioneiro1, Prisioneiro2, Guarda1]

[7] QC3 - Detentores dos detonadores:
      Detonador_Civil         -> Civil1
      Detonador_Prisioneiros  -> Guarda1

[8] QC4 - Detonadores acionados:
      Detonador_Civil         -> false
      Detonador_Prisioneiros  -> false
      Bombas com explosao impedida: true, true
      Embarcacoes salvas: true, true

[9] QC5 - Passageiros sobreviventes (6):
      * Civil1
      * Civil2
      * Policial1
      * Prisioneiro1
      * Prisioneiro2
      * Guarda1

[10] QC6 - Passageiros mortos (0): []

[11] QC7 - Hipotese do Coringa confirmada? false

================================================
  Execucao concluida com sucesso.
================================================
```

Listing 1: Saída da execução — JokerTest-Ontology.